

Gauntlet — Joypad Harness Planning Document

A runtime safety harness that governs any AI game-playing agent. The worker plays a game through a controller interface; the harness decides what inputs are legal, measures real progress, carries game state in and out, and raises structured alarms. When the agent gets stuck, the harness feeds corrective feedback back in and the agent changes course. The harness, not the agent, is what’s evaluated.

The four pillars (each separate from the worker)

Pillar	Responsibility
 Guardrails	Declared input rules — allowed buttons, max hold, rate limits. Checked <i>before</i> execution.
 Checkpoints	Pass/fail on progress from game state (position, score, lives). Persisted to SQLite.
 Material handling	Capture/normalize game state; persist checkpoints; replay from any point.
 Alarms	<code>{type, severity, context, recommended_action}</code> — drive escalation and recovery.

The core: behavior changes from feedback

The graded “must.” Loop: `read state → agent.decide(state) → guardrails validate → execute window → checkpoint pass/fail → on FAIL: feedback hint + alarm → loop`. When `FORWARD_PROGRESS` fails, the harness doesn’t silently retry — it injects a hint (“stuck 120 frames, try JUMP”) into the agent’s next decision and the same agent clears the obstacle. Stuck → told why → adapts → succeeds: the demo’s hero beat. (The agent decides an *intent* over a ~60-frame window, not per frame — that’s what makes an LLM-in-the-loop real-time feasible.)

Tech stack

Python + FastAPI harness service and control loop. **jsnes** (browser NES emulator) gives RAM access for exact game state, so checkpoints are objective and the agent reads structured state, not pixels. A **WebSocket bridge** renders the game live in the browser while the harness streams events. The **worker** uses the Anthropic SDK with Claude Haiku 4.5 (fast/cheap reflex) and structured-output actions, plus a scripted bot. **SQLite** holds checkpoint and state persistence for replay.

Dashboard, worker independence & escalation

The dashboard *is* the deployed URL: game on the left, the four pillars firing on the right. Any worker (`decide(state) → action`) drops in unchanged — Claude, GPT, or scripted bot; repeated failures trip a `STOP` that escalates to a human. **Bonus:** live worker swap.

Demo: Mario hits a pipe → `FORWARD_PROGRESS` fails → `AGENT_STUCK (HIGH)` → jump hint fed back → same agent clears it → live guardrail block → human escalation → worker swap → checkpoint replay.